

Lab 6 - 1A

Jibon Naher¹ and Samnun Azfar²

¹Lecturer, CSE

²Junior Lecturer, CSE

December 22, 2025

General Instructions

1. Use the PC from the lab.
2. Submit your mobile at the front.
3. Turn off your internet connection.
4. Your time is 50 minutes.

Part A (Mutex): Priority Logging System

Problem Statement

Concurrent threads print High or low priority log messages. High-priority messages **must always** be processed/printed before Low-priority ones, regardless of arrival time within the **critical window**.

Input (Thread Actions)

- Thread 1 (Low): Prints "Network OK"
- Thread 2 (High): Prints "Disk Failure"
- Thread 3 (Low): Prints "User Login"
- Thread 4 (High): Prints "Power Surge"

Constraints

- Use **one mutex**.

Expected Output (Buffer Dump)

```
[HIGH] Disk Failure
[HIGH] Power Surge
[LOW] Network OK
[LOW] User Login
```

PART B: Semaphore: Studying Students Problem

Objective

Simulate N students sitting around a circular table, each alternating between reading and writing. Each student requires two shared pens (one from their left and one from their right) to write. The program must ensure: **Mutual Exclusion**: No two neighboring students can write simultaneously, **Deadlock Prevention**: The system must not enter a state where no one can proceed, **Safe Access**: Shared pens and states must be accessed using mutexes and semaphores.

Scenario

There are N students, numbered 1 to N . Each student alternates between three states: **Reading** (Non-critical section), **Waiting for pens** (Trying to enter critical section), **Writing** (Critical section).

Resource Sharing Model:

- Student i shares one pen with student $(i - 1)$ and one with student $(i + 1)$.
- In a circular arrangement, student 1's left neighbor is student N .

Partially Provided Code

The following thread function defines the high-level behavior of each student:

student_thread.c

```
1 // Thread function for each student
2 void* student(void* num) {
3     while (1) {
4         int i = *(int*)num;
5         sleep(1); // Reading time
6         take_pens(i); // Try to acquire both pens to write
7         sleep(2); // Writing duration
8         put_pens(i); // Return pens and go back to reading
9     }
10 }
```

Your Tasks

1. Define Constants and Shared Variables

Define necessary constants (e.g., N) and shared variables (e.g., sem_t mutex) needed.

2. Implement Helper Functions

- void test(int i): Checks if the i^{th} student can start writing. A student can only write if they are in the *Waiting* state and neither of their neighbors is currently *Writing*.
- void take_pens(int i): Marks the student as *Waiting* and attempts to acquire both pens. This must use the global mutex to protect state changes.
- void put_pens(int i): Marks the student as *Reading* and releases both pens. It should also call test() on the left and right neighbors to signal them if they are waiting.
- Write the main() Function.